Chapter 02 -- Building Your First Ontology in Protégé

After establishing the conceptual foundations of ontology engineering in Chapter 01, we now move into the practical world of ontology construction inside Protégé. This chapter marks the transition from semantic theory into hands-on ontology modeling.

The goal of this chapter is not merely to "click through a tool," but to understand how semantic structures are engineered systematically. In traditional software engineering, developers write executable code. In ontology engineering, architects build executable meaning.

This distinction is fundamental.

The Pizza ontology may appear simple on the surface, but it introduces modeling patterns used throughout enterprise semantic systems, knowledge graphs, and AI-driven architectures.

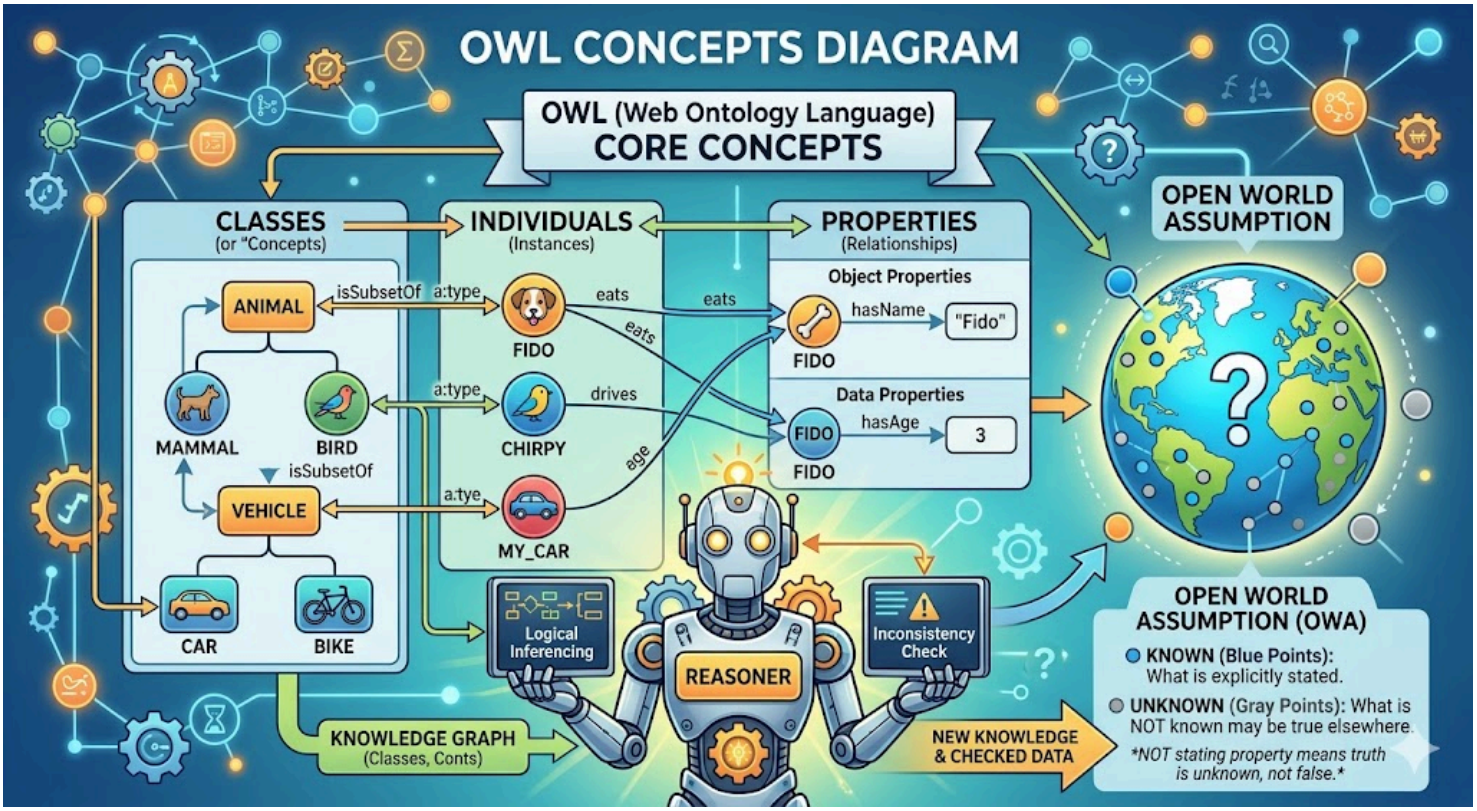
This chapter is primarily based on the ontology construction sections from Michael DeBellis' revised Pizza Tutorial for Protégé 5.5.

2.1 From Theory to Construction

In Chapter 01, we introduced the core OWL concepts:

- Classes
- Individuals
- Properties
- Reasoners
- Open World Assumption

A quick AI-generated picture below brings them together systematically:



Now we begin building actual semantic structures.

This is where ontology engineering starts to feel very different from traditional modeling disciplines.

In a traditional diagramming tool, creating a class hierarchy is often just a visual activity. In Protégé, however, every modeling action contributes to a machine-processable semantic structure.

This means that:

- class hierarchies have logical implications
- property definitions affect reasoning behavior
- restrictions become reasoning-enabled constraints, and
- inference engines can derive new knowledge automatically

As Michael DeBellis emphasizes throughout the tutorial, ontology engineering should be approached as semantic knowledge modeling rather than merely graphical editing.

2.2 Understanding the Protégé Workspace

When Protégé is launched for the first time, many new users feel overwhelmed by the interface.

This is normal.

Protégé is not designed as a lightweight diagramming application. On the contrary, it is a **semantic engineering environment**.

The workspace contains multiple perspectives for editing different ontology components:

- Classes
- Object Properties
- Data Properties
- Individuals
- Annotations
- Reasoner views

- Query views
- Rule views

One of the most important concepts for beginners to understand is that Protégé does not separate "diagramming" from "logic".

Everything visible in the interface ultimately contributes to OWL axioms underneath.

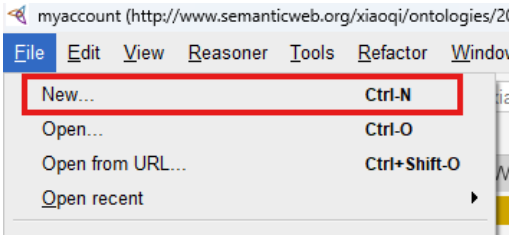
This is why ontology engineering feels closer to knowledge programming than traditional modeling.

2.3 Creating a New OWL Ontology

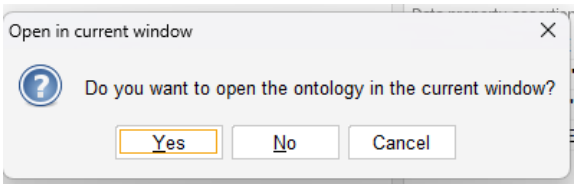
The first practical step is creating a new ontology project.

During this chapter's writing, I'm using Protégé v5.6.9.

Inside Protégé: `File > New...` :



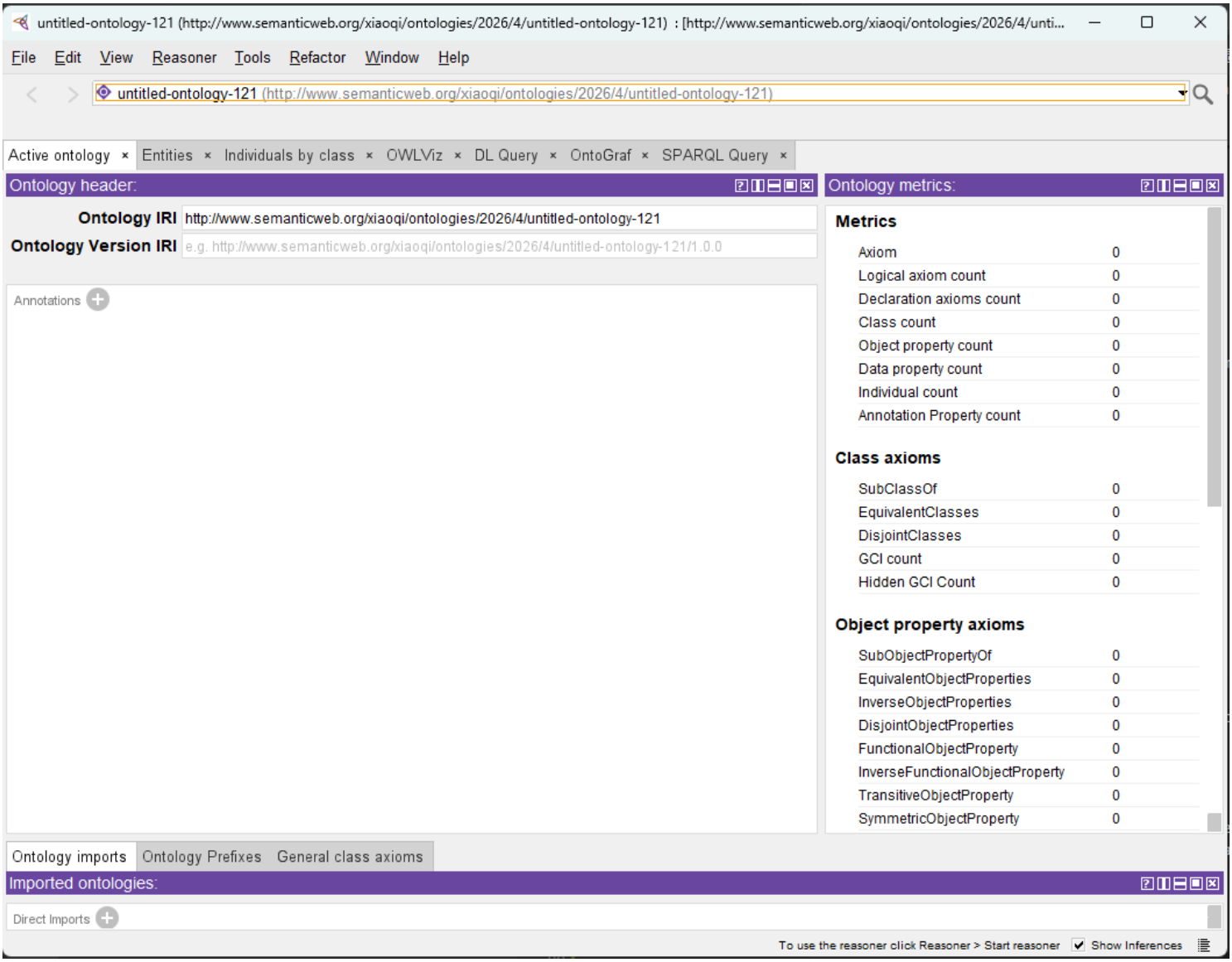
If you're having existing ontology file opened in Protégé, you will get below pop-up with question:



If you choose `Yes` (default), your current ontology file will be closed and your new ontology project will occupy current window, ensure you save any previous changes before losing them.

Or, you may choose `No` so a new window will be opened for your new ontology project, your existing opened ontology will not be impacted.

You then can see the new ontology project screen:



When you want to save the project, Protégé then asks for:

- Ontology IRI
- Document storage location
- RDF/XML or OWL serialization format

At this stage, beginners often underestimate the importance of ontology IRIs - Internationalized Resource Identifier.

As defined by W3C, "*Each ontology may have an ontology IRI, which is used to identify an ontology.*" An ontology IRI acts as the global semantic identifier for the ontology.

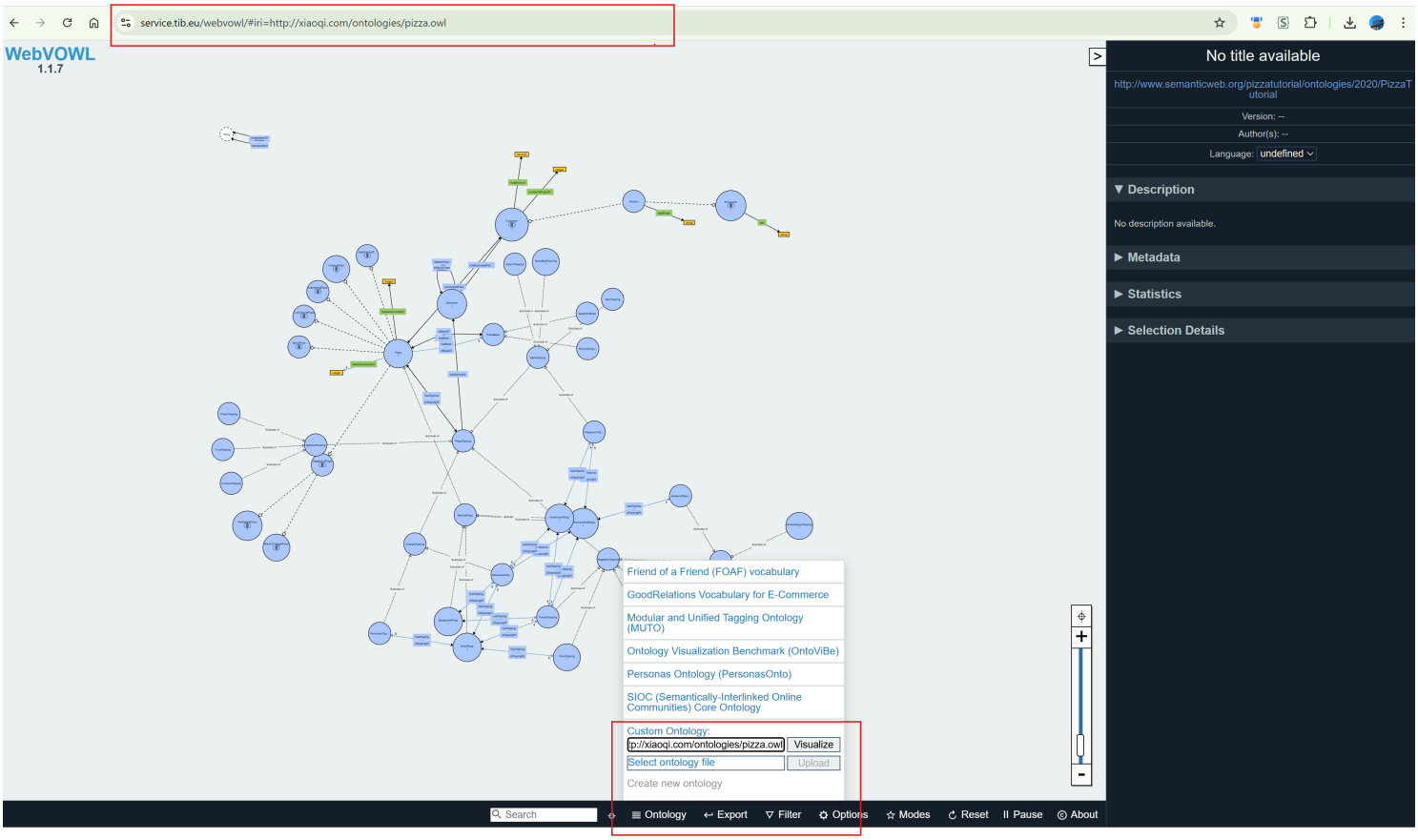
Unlike filenames or database table names, ontology IRIs are intended to be globally unique identifiers in semantic ecosystems.

For example:

```
# IRI for pizza.owl
http://xiaoqi.com/ontologies/pizza.owl

# My new ontology project IRI (local)
http://www.semanticweb.org/xiaoqi/ontologies/2026/4/untitled-ontology-121
```

Using WebVOWL - the OWL Visualization Tool, you can paste above `pizza.owl` IRI then visualize it for now:



This is conceptually similar to namespaces in programming languages, but with web-scale semantic interoperability.

In enterprise environments, ontology naming strategies become critically important because ontologies may later integrate across:

- business domains,
- data catalogs,
- APIs,
- knowledge graphs,
- AI agents, and
- semantic search platforms.

This is one of the first points where ontology engineering begins to intersect with enterprise architecture governance.

2.4 The Importance of Ontology Naming Conventions

One of the subtle but important lessons from the Pizza tutorial is disciplined naming.

Ontology projects quickly become unmanageable if naming conventions are inconsistent.

Michael DeBellis repeatedly emphasizes semantic clarity and hierarchy organization throughout the tutorial.

Good ontology naming should satisfy several criteria, as below:

- Human-readable
- Machine-processable
- Semantically meaningful
- Consistent across domains
- Stable over time

For example in our `Pizza.owl` :

```
Pizza
PizzaTopping
VegetarianPizza
SpicyPizza
```

These names are concise and semantically clear.

In enterprise ontology projects, naming conventions become even more important because ontologies may evolve into shared organizational semantic standards.

Within EKA, naming discipline is essential because ontology entities later become:

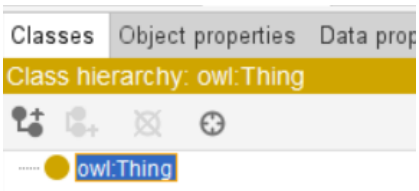
- graph nodes
- semantic APIs
- AI context objects
- digital twin entities
- and executable business semantics

Poor naming eventually creates **semantic debt** and chaos across the organization.

This is analogous to **technical debt** in software engineering.

2.5 Understanding `Thing` : The Root of All Classes

Every OWL ontology begins with a universal root class: `owl:Thing`



All classes ultimately inherit from `Thing` .

This concept initially appears trivial, but it represents an important philosophical principle in ontology engineering.

> OWL assumes that all modeled entities exist within a universal semantic space.

For example:

```
Thing
 |-- Pizza
 |-- PizzaTopping
 |-- PizzaBase
 |-- Spiciness
```

Unlike relational databases where tables are independent structures, OWL organizes concepts into a globally connected semantic universe.

This is one reason ontology systems integrate naturally with knowledge graphs.

2.6 Creating the Core `Pizza` Taxonomy

The next step involves building the foundational class hierarchy.

From Wikipedia, **Taxonomy** is a practice and science concerned with classification or categorization.

The Pizza ontology begins with several top-level conceptual categories:

```
Pizza
PizzaTopping
PizzaBase
Spiciness
```

These represent the primary semantic domains within the ontology.

At this stage, ontology engineering resembles taxonomy construction.

However, ontologies extend far beyond simple taxonomies.

An ontology additionally defines:

- semantics
- constraints
- properties
- logical restrictions
- inference behavior
- and reasoning rules

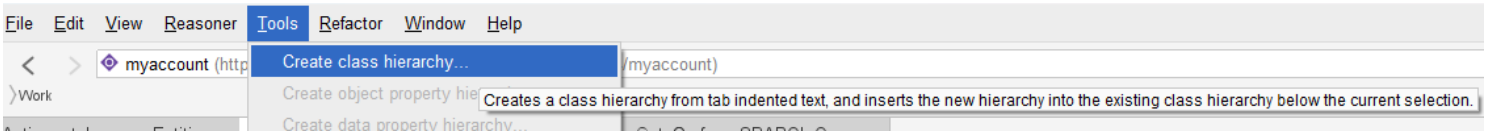
This distinction is critical.

Many enterprise "ontologies" are actually only taxonomies because they lack formal semantics.

2.7 Building Class Hierarchies Systematically

One of the most powerful productivity features demonstrated in the Pizza tutorial is the "Create Class Hierarchy" wizard.

In Protégé, from menu [Tools] > [Create class hierarchy...]



Instead of manually creating classes one by one, Protégé allows ontology engineers to rapidly construct semantic hierarchies using indentation-based structure.

For example:

```
PizzaTopping
  CheeseTopping
  MeatTopping
  SeafoodTopping
  VegetableTopping
```

> Tip: you may create those structure in one TXT file upfront, then paste into the wizard.

This approach provides several advantages:

- Faster ontology construction
- Better structural consistency
- Easier semantic organization
- Able to see `disjoint` in one go (see 2.8 below)
- Reduced modeling errors

In enterprise-scale ontology projects containing thousands of concepts, hierarchical discipline becomes essential.

Without strong hierarchy governance:

- semantic duplication emerges
- inheritance becomes inconsistent
- and reasoning performance degrades

Ontology engineering therefore requires architectural thinking, not merely modeling skills.

2.8 Disjoint Classes: Preventing Semantic Contradictions

An extremely important concept introduced during hierarchy creation is **class disjointness**.

For example:

```
MeatTopping
VegetableTopping
SeafoodTopping
```

These categories should logically be mutually exclusive.

A topping cannot simultaneously be both:

- MeatTopping
- and VegetableTopping

Protégé allows classes to be declared as disjoint.

This enables reasoners to detect contradictions automatically.

For example, if an individual were incorrectly classified as both:

```
HamTopping
AND
VegetableTopping
```

Since `HamTopping` is one type of `MeatTopping`, the reasoner would identify an inconsistency.

This capability becomes enormously important in enterprise environments.

The Disjointness supports:

- data quality enforcement
- semantic governance
- AI validation
- and knowledge consistency checking

Within EKA, this is one of the first places where architecture becomes executable.

The ontology no longer merely describes the domain, it actively validates semantic correctness.

2.9 Primitive vs Defined Classes

One of the most intellectually important concepts in OWL modeling is the distinction between:

- Primitive Classes
- Defined Classes

2.9.1 Primitive Class

A primitive class only specifies necessary conditions.

For example:

```
Pizza
```

may simply be declared as a subclass of:

```
Food
```

without fully defining what constitutes a pizza.

2.9.2 Defined Class

A defined class, however, specifies both:

- necessary conditions
- and sufficient conditions

> You may find similar terms in Sets Theory, correct, they're inherited from that theory indeed.

For example:

```
VegetarianPizza
  ≡ Pizza AND (hasTopping only (CheeseTopping or VegetableTopping))
```

This means the reasoner can automatically infer membership.

If a pizza satisfies the logical conditions, it becomes classified automatically.

This is one of the defining characteristics of semantic systems.

Traditional systems rely heavily on manual categorization, while ontology systems support computational classification.

2.10 Why Automated Classification Matters

This concept may initially seem academic, but it becomes transformative at enterprise scale.

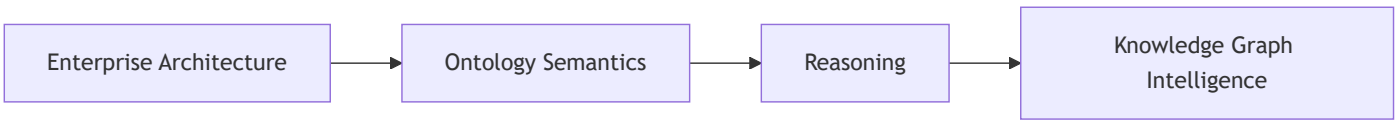
Consider a large organization containing:

- thousands of applications,
- millions of data entities,
- thousands of APIs, and
- hundreds (or possible thousands as well) of business capabilities.

Manually classifying all semantic relationships becomes impossible.

Ontology reasoning introduces automation into semantic governance.

Within EKA, automated classification enables:



This is one of the major strategic advantages of semantic architecture over static repositories.

Note

Automated classification makes the ontology **reasoning-enabled**. However, reasoning alone does not make the ontology **executable**. True executability within the EKA framework requires **triggers (Θ)** and **execution action (Φ)** that respond to semantic outcomes.

2.11 Ontology Engineering vs Diagramming

A common beginner's mistake is treating Protégé like Visio or UML modeling software.

This is fundamentally incorrect.

In diagramming tools:

- relationships are often visual only
- semantics remain informal
- and diagrams rarely execute

While in ontology engineering:

- semantics are formalized
- relationships are machine-readable
- and logic becomes executable

This difference is profound.

Ontology engineering sits much closer to software engineering and knowledge programming than to traditional architecture diagramming.

Protégé therefore represents a semantic compiler environment rather than merely a drawing tool. (In fact, you don't have actual "drawing" activities in Protégé.)

This distinction becomes central within EKA because the ultimate goal is not documentation.

The goal is executable knowledge.

2.12 Early Modeling Discipline

One of the most valuable lessons from the Pizza tutorial is modeling discipline.

Good ontology engineering requires:

- semantic clarity,
- naming consistency,
- hierarchy governance,
- logical rigor,
- and iterative refinement

New ontology engineers often attempt to model everything immediately.

Experienced ontology architects instead:

1. establish clean top-level structures,
2. define semantic boundaries,
3. create reusable patterns,
4. and evolve the ontology incrementally.

This mirrors good enterprise architecture practices.

Ontology engineering is therefore both:

- a technical discipline,

- and an architectural discipline.

2.13 The Emerging Semantic Architecture Mindset

At this stage in the learning journey, you should begin recognizing a major conceptual transition.

Traditional enterprise systems are largely:

```
Data-centric
```

Ontology systems are

```
Meaning-centric
```

This shift has enormous implications for AI systems.

Modern intelligent systems increasingly require:

- semantic interoperability,
- contextual understanding,
- knowledge reasoning,
- and machine-readable meaning

Ontology engineering provides the formal semantic layer necessary for this evolution.

This is why ontology, knowledge graphs, and semantic AI are rapidly converging across modern enterprise architecture.

2.14 Chapter 02 Summary

In this chapter, we began constructing the foundational structure of the Pizza ontology inside Protégé.

We have explored together:

- the Protégé workspace,
- ontology project initialization,
- ontology IRIs,
- semantic naming conventions,
- class hierarchy creation,
- disjoint classes,
- primitive vs. defined classes,
- and automated classification concepts.

More importantly, we examined how ontology engineering fundamentally differs from traditional diagramming approaches.

Protégé was introduced not simply as a modeling tool, but as a semantic engineering environment capable of creating executable knowledge structures.

The chapter also reinforced the EKA perspective that ontology serves as the semantic transformation layer between enterprise architecture and executable intelligence systems.

2.15 Protégé Operations (Skills) Learned

During this chapter, you learned how to:

- Create a new ontology
- Configure ontology IRIs

- Navigate the Protégé interface
- Create top-level classes
- Build class hierarchies
- Use the Create Class Hierarchy wizard
- Define semantic taxonomies
- Apply disjointness
- Organize ontology structures systematically

2.16 EKA Connection

Within EKA, this chapter represents the transition from static architecture modeling toward executable semantic structure.

Traditional enterprise repositories often contain disconnected diagrams.

Ontology introduces:

- formal semantics,
- machine readability,
- automated reasoning,
- and semantic consistency validation.

The class hierarchies created in this chapter are not merely documentation artifacts.

They are the foundational semantic structures that later evolve into:

- enterprise knowledge graphs,
- AI semantic layers,
- digital twins,
- and reasoning-enabled architectural intelligence -- the prerequisite for truly executable knowledge systems.

2.17 Knowledge Graph Perspective

Knowledge graphs require semantically meaningful entities and relationships.

The ontology structures created in this chapter provide the semantic backbone necessary for graph intelligence.

> **Graph Database Loading Paths for OWL Ontologies**
>
> OWL ontologies can be loaded into graph databases, but the implementation path depends on how much semantic fidelity you need to preserve:
>
> 1. **Native RDF Graph Databases (Recommended for EKA)**: AllegroGraph, GraphDB, Stardog, RDFox, Amazon Neptune (RDF mode), and Apache Jena/Fuseki. These systems store data natively as RDF triples and can **load OWL ontologies directly**, preserving full formal semantics, global IRI identity, and reasoning capabilities. This is the recommended choice for the EKA K layer.
> 2. **Property Graphs (For Specialized Performance Scenarios)**: Neo4j, TigerGraph, JanusGraph, and Amazon Neptune (property graph mode). These systems use internal, non-semantic element IDs. Loading an OWL ontology into a property graph is a **lossy transformation**, not a direct mapping. You lose OWL semantics, reasoning capabilities, and IRI-based global interoperability. Use this path only for specialized, performance-intensive graph traversal scenarios.

For example:

```
Pizza
  ↓ hasTopping
CheeseTopping
```

can later become graph nodes (entities) and graph edges (relationships) enriched within formal semantics.

Ontology therefore serves as the semantic schema layer for knowledge graphs.

Without ontology:

- graph semantics become ambiguous,
- relationships lose formal meaning,
- and reasoning capability becomes limited.

2.18 Key Concepts

Term	Definition
OWL Ontology	A formal semantic model built using the Web Ontology Language (OWL), consisting of classes, properties, and individuals that represent knowledge in a machine-readable and logically rigorous form.
Protégé	An open-source ontology editor and semantic modeling platform used to create, visualize, and reason over OWL ontologies. It serves as the primary tool for hands-on ontology engineering in this book.
owl:Thing	The universal superclass from which all OWL classes inherit.
Taxonomy	A hierarchical classification structure organizing concepts into parent-child relationships.
Ontology	A semantic model that includes taxonomy, properties, restrictions, semantics, and reasoning logic.
Class	A category or type of thing in an ontology (e.g., `Pizza`, `PizzaTopping`). Classes can be organized into hierarchies to represent semantic inheritance.
Subclass	A class that inherits characteristics from a parent class. For example, `VegetarianPizza` is a subclass of `Pizza`.
Disjoint Class	Classes that are declared mutually exclusive — an individual cannot belong to two disjoint classes simultaneously (e.g., `MeatTopping` and `VegetableTopping`).
Primitive Class	A class that defines only necessary conditions. The reasoner cannot automatically classify individuals into primitive classes based on logical definitions alone.
Defined Class	A class that defines both necessary and sufficient conditions. The reasoner can automatically classify individuals into defined classes when the conditions are satisfied.
Ontology IRI	An Internationalized Resource Identifier that uniquely identifies an ontology. It serves as the global semantic identifier for the ontology, enabling interoperability and reuse across systems.
Taxonomy	A hierarchical classification structure that organizes concepts into parent-child relationships (e.g., `PizzaTopping` → `CheeseTopping`). A taxonomy provides the structural foundation for an ontology.
Necessary Condition	A condition that must be true for an individual to belong to a class. For example, all `Pizza` must have a base.
Sufficient Condition	A condition that, if satisfied, is enough to classify an individual into a class. For example, if a pizza contains only vegetable toppings, it can be inferred to be `VegetarianPizza`.
Automated Classification	The process by which a reasoner automatically assigns individuals to defined classes based on logical conditions, without manual intervention.
Semantic Debt	The cumulative consequences of poor ontology design decisions, such as inconsistent naming, missing constraints, or improper hierarchy. Analogous to technical debt in software engineering.
EKA (Executable Knowledge Architecture)	An architectural framework that transforms formal knowledge into machine-executable intelligence through a structured pipeline: Diagrams → Meta-Models → Ontologies → Knowledge Graphs → Executable Intelligence.

Term	Definition
Knowledge Graph	A graph-structured knowledge base that represents entities as nodes and relationships as edges, enriched with formal semantics from an ontology.
Reasoner	A logic engine that checks consistency, infers implicit relationships, and automatically classifies individuals based on defined axioms.

2.19 Next Chapter (03) Preview

In the next chapter, we will install Protégé and become familiar with the ontology engineering workspace used throughout the Pizza OWL tutorial series.

Readers will learn how to:

- install Protégé properly,
- navigate the Protégé interface,
- understand the major workspace areas,
- explore ontology editing perspectives,
- and begin developing the mindset of semantic engineering.

This step establishes the operational foundation for all future ontology modeling activities in the book.

Protégé is not simply a modeling application.

It is the primary semantic engineering environment where ontology structures are transformed into machine-readable knowledge models. Within the EKA framework, this represents the beginning of hands-on ontology engineering practice and the transition from conceptual understanding into executable semantic modeling.

2.20 Reference

- Protégé Pizza Repository: https://github.com/yasenstar/protege_pizza
- WebVOWL project in GitHub: https://github.com/VisualDataWeb/WebVOWL
- EKA Official Website: https://xiaoqi.com

2.21 Demo Video for this Chapter

YouTube Demo Video - Chapter 02: https://youtu.be/eWx9_zJkiUY

Last updated: 2026-09-02, thanks for @Jah-yee's pull request